PI Protocol: Fullstack Privacy

Logic Chaining & Feature Implementation

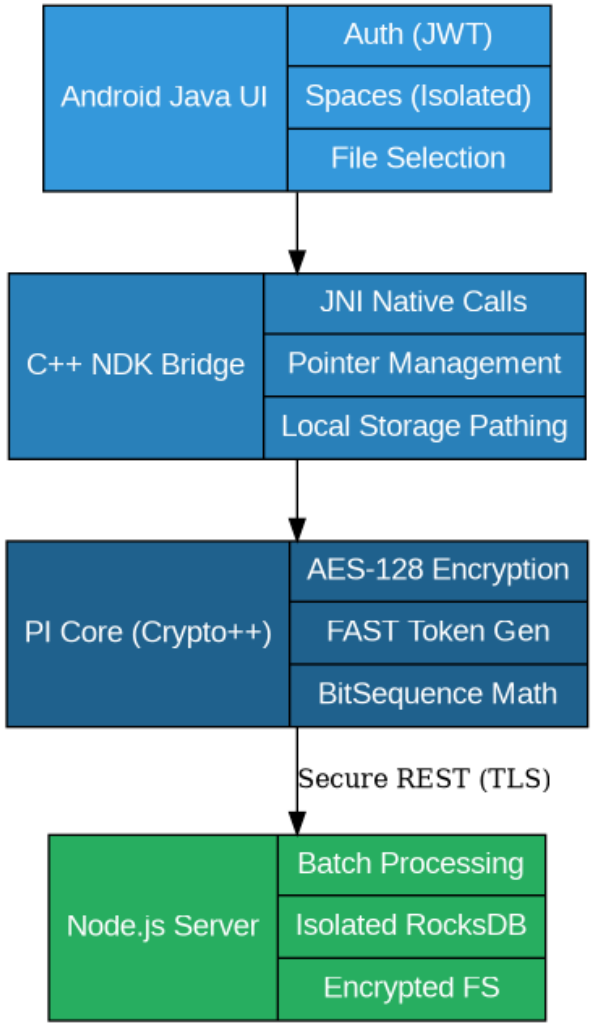
Deep Dive Overview

March 2026

The Logic Chain: Data Journey

How data moves safely:

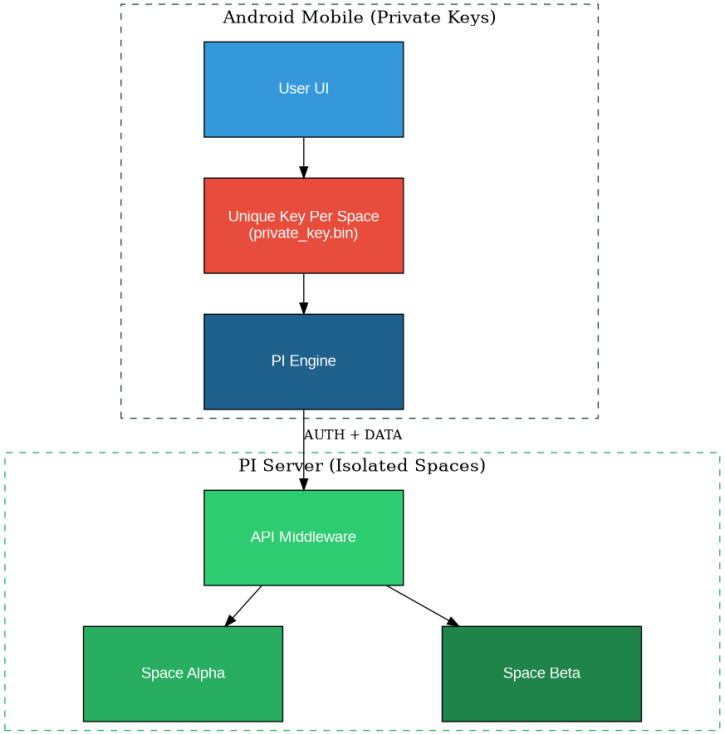
- **Selection**: Java layer selects files/keywords.
- **Processing**: NDK invokes C++ PI Engine.
- **Tokenization**: FAST scheme generates 200k tokens.
- **Batched Sync**: Tokens sent in chunks of 5,000.



Isolated Secure Spaces

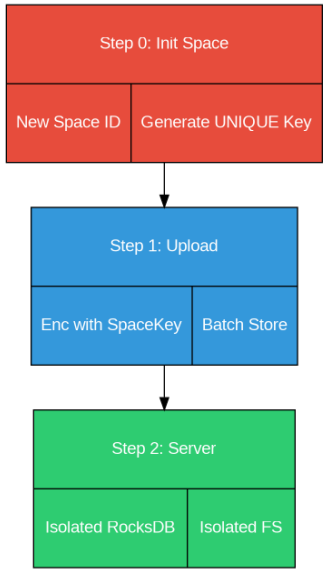
Zero-Trust Isolation:

- **Physical Separation**: Every space has a unique RocksDB + FS folder.
- **Unique Keys**: Private keys generated per-space; never leave the phone.
- **Session Security**: JWT authentication protects all space interactions.

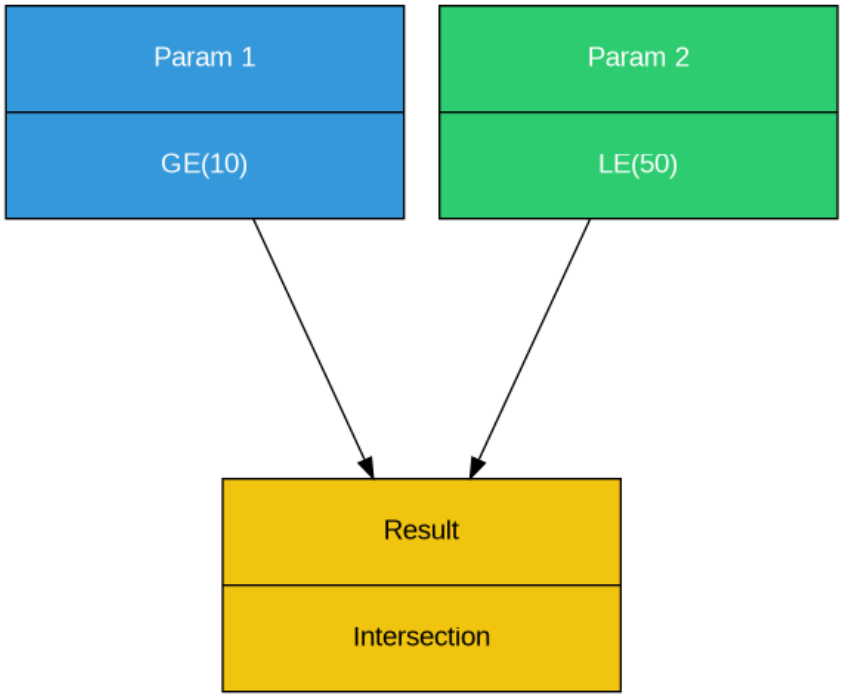


Advanced Update & Metadata

- **Extension Preservation**: Encrypts `doc.pdf` to `ID0.pdf`. The server knows the extension (so storage works) but not the content.
- **Payload Batching**: The App prevents Out-Of-Memory (OOM) network crashes by chunking uploads into exactly 5,000-token arrays and 5-file bundles per HTTP POST.
- **Local State Management**: The phone tracks indexing state locally in a mini-RocksDB to prevent token collision.



Private Search Strategy



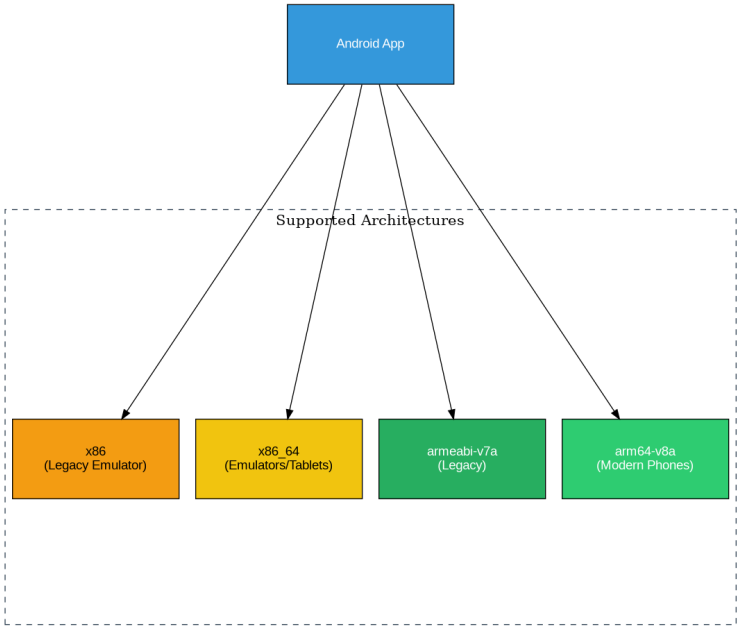
How Range Queries Work:

- **Trapdoors**: App generates 2 secure search tokens.
- **Server Side**: Returns encrypted match sets.
- **Intersection**: `BitSequence` logic on phone finds the final result.

Universal Android Support

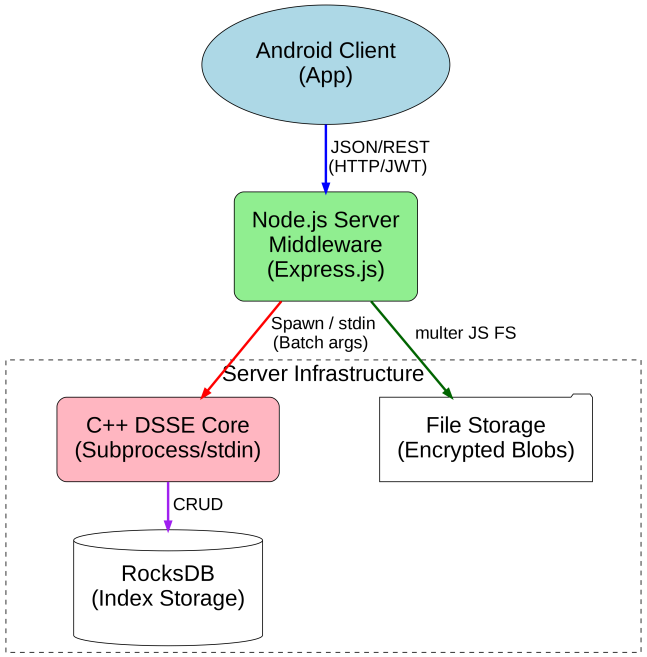
Multi-Core Stability:

- Supports **arm64-v8a**, **v7a**, **x86**, and **x64**.
- High-speed static C++ libraries linked via CMake.
- Hardened JNI to prevent native crashes.



Server Architecture: Node.js Middleware

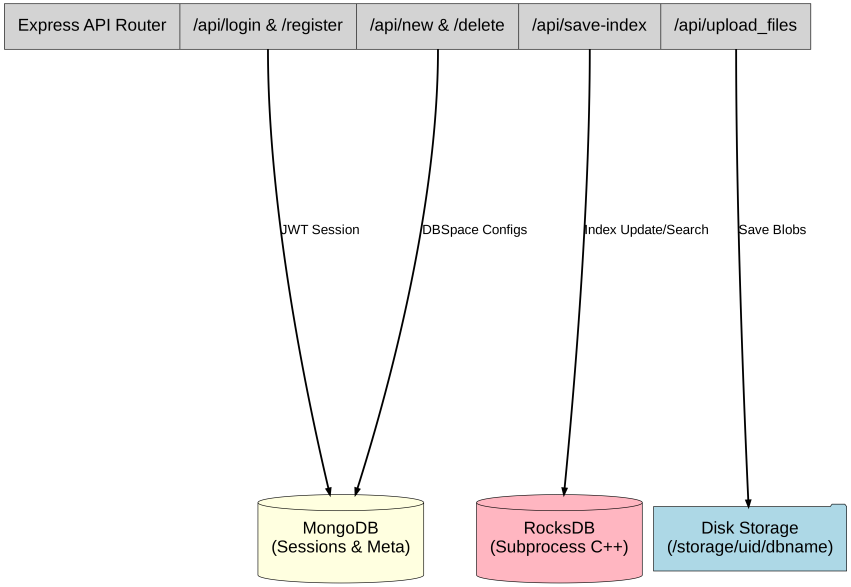
- **Role**: Acts as a secure REST middleware bridging the Android client with the C++ DSSE Core and File Storage.
- **Decoupling**: Abstracts away complex command-line native executions (RocksDB) into predictable, standard REST calls.
- **Native Integration**: Spawns `dsse_server` child processes, feeding large batch updates via `stdin` streams to bypass OS command-line limits.



API Endpoints & Security

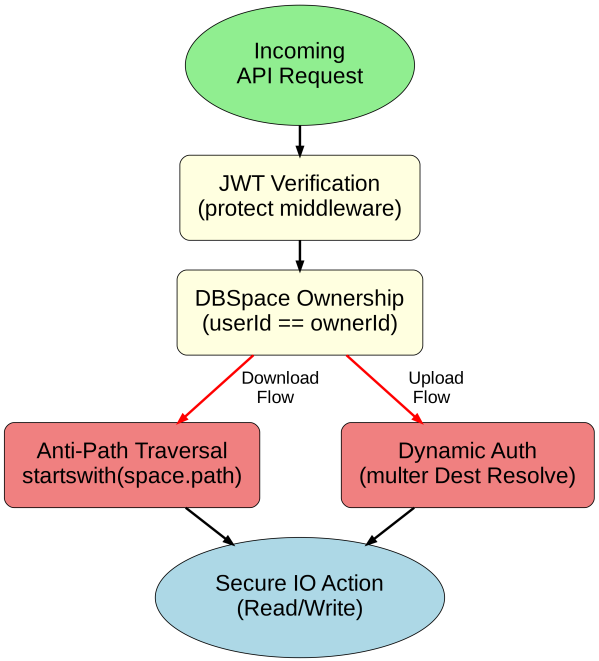
Endpoint Structure & Purpose:

- **Auth** (`/login`): JWT-based sessions in MongoDB.
- **Space** (`/new`): Provisions isolated `DBSpace`.
- **Index** (`/bulk-save`): Interacts with RocksDB for search/update.
- **File** (`/upload`): Handles encrypted document blobs.



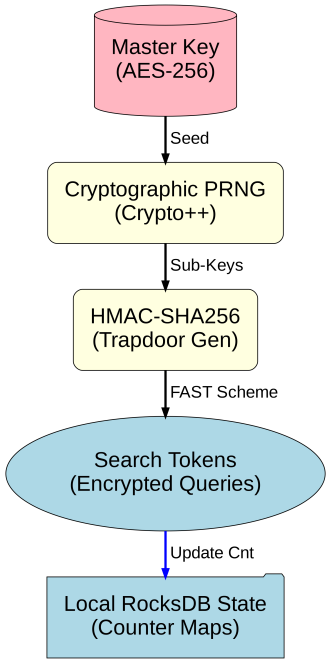
Security & Edge Case Handling

- **Zero-Trust**: `protect` middleware validates JWT AND ownership.
- **Immutable Locks**: Upon completion, the App triggers `/lock-space`. The API irrevocably sets `isLocked=true` in Mongo, physically preventing hackers from re-uploading and destroying the RocksDB index.
- **Anti-Path Traversal**: Verifies `startsWith()` against `../` attacks.
- **Dynamic Storage**: `multer` safely resolves path before bytes read.



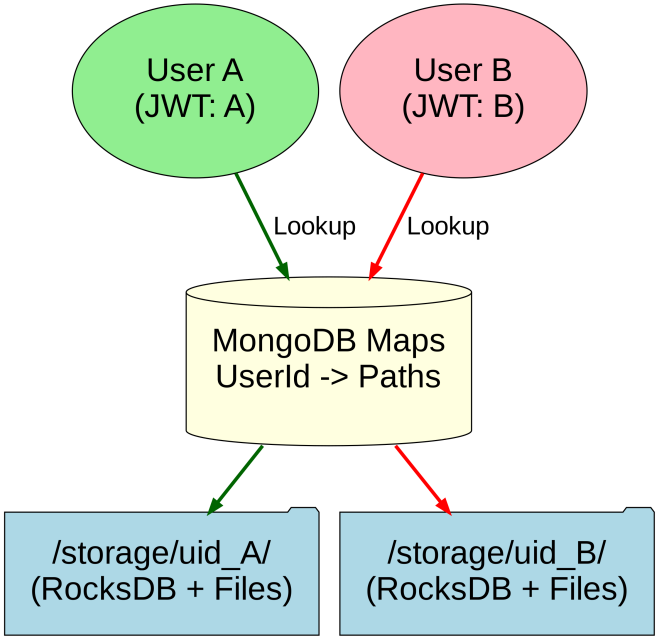
Cryptographic Core: The FAST Scheme

- **C++ & NDK**: High performance native cryptographic routines using `Crypto++`.
- **Master Key Derivation**: Initial AES-256 keys generate subsequent sub-keys via Pseudo-Random Number Generators (PRNG).
- **HMAC-SHA256**: Encrypts both the file paths and search keywords into Trapdoor Tokens.
- **Local Counters**: Phone retains state integers via a local RocksDB preventing duplicate tokens.



Multi-Tenant Storage Isolation

- **Complete Compartmentalization**: Each user's database is a 100% physically separate folder on the Linux server filesystem.
- **JWT → UID Mapping**: The API extracts the authenticated UID from the Bearer Token, never trusting the client's payload.
- **MongoDB Lookup**: The DBSpace schema strictly links the requested database name to the verified UID to extract the true absolute path (`/storage/uid_XYZ/dbname/`).



App Functionality Summary

1. **Login**: Secure JWT session establishment.
2. **Update**: Batch token generation & file encryption.
3. **Search**: Range query intersection on encrypted data.
4. **Recovery**: Secure local decryption of search results.

DEMO